

MANUAL TÉCNICO

Arquitectura · Firmware · BOM · Mantenimiento

1. TEORÍA DE OPERACIÓN Y ARQUITECTURA DEL SISTEMA

LANCEA implementa una arquitectura **Store-and-Forward** basada en una Máquina de Estados Finitos (FSM) de 3 estados. Esta arquitectura garantiza determinismo temporal en la adquisición de datos sin bloquear el servidor web embebido ni la comunicación I²C con el IMU.

■ Esquemático de conexiones — KiCad 7.0

La figura siguiente muestra el diagrama eléctrico completo del sistema LANCEA. U1 = XIAO ESP32-C3-DIP (controlador principal) · U3 = IMU 2472/BNO055 · U4 = MicroSD (interfaz SPI) · BZ1 = Buzzer · SW1 = pulsador de usuario · J2 = Terminal de tornillo 1×6 para expansión/depuración.

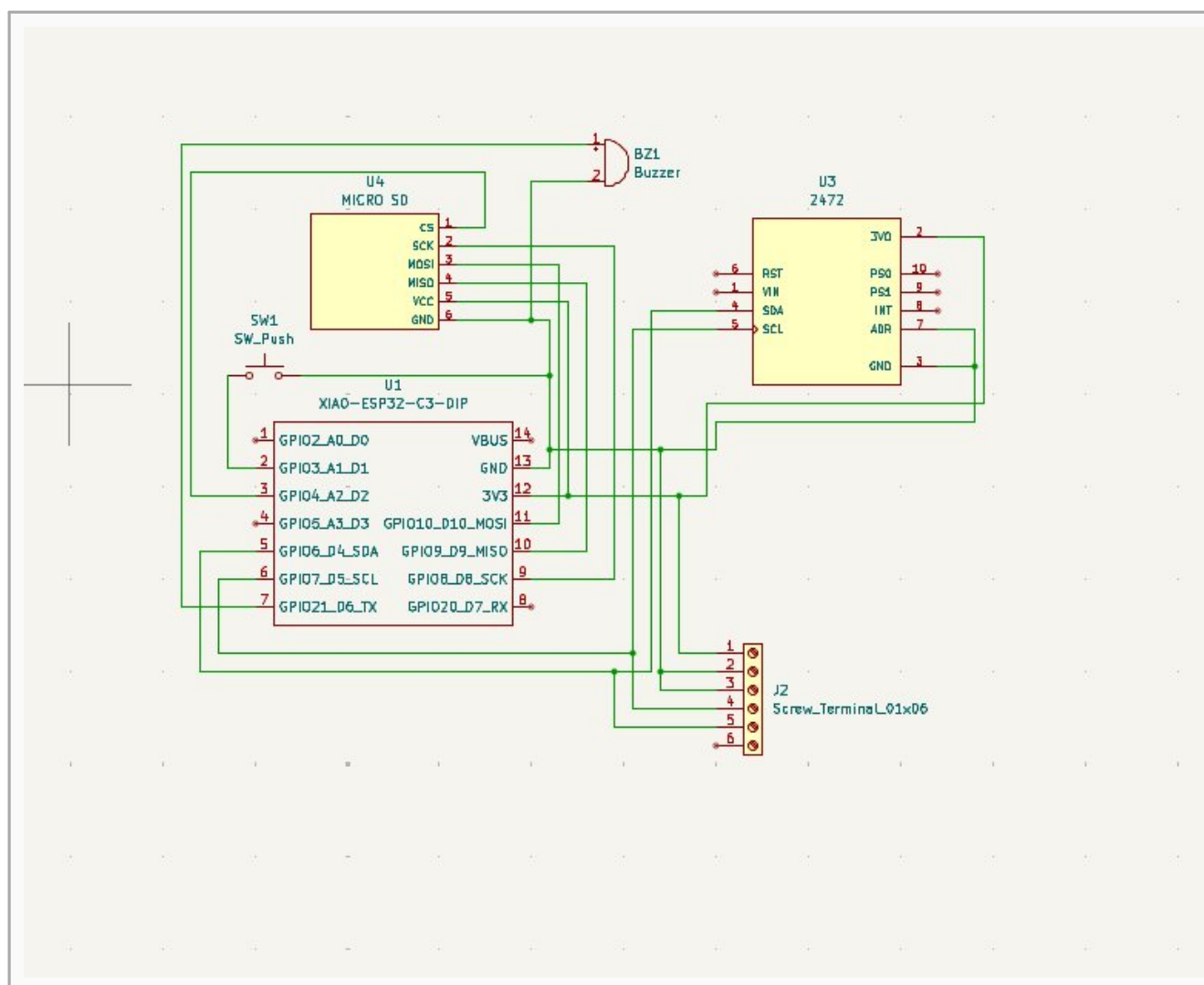


Figura 1. Esquemático de conexiones eléctrico del sistema LANCEA — XIAO ESP32-C3 (U1) · MicroSD (U4) · IMU 2472/BNO055 (U3) · Buzzer (BZ1) · Terminal de tornillo (J2) · Pulsador (SW1). Generado con KiCad 7.0.

Máquina de estados finitos (FSM) del firmware:

ESTADO	CONDICIÓN DE ENTRADA	ACCIONES EJECUTADAS	CONDICIÓN DE SALIDA
IDLE	Estado inicial al encender	Lee J(t) del BNO055 cada 10ms. LED verde parpadeante. Servidor web activo.	$ J(t) > \text{JERK_THRESHOLD}$ (50 m/s³) Y $ a > \text{ARMING_ACCEL}$ (15 m/s²)
IMPULSO	$ J > \text{umbral configurado}$	Integración Euler explícita de velocidad en 3 ejes a 100 Hz. LED rojo fijo. Buffer RAM llenando.	$ J < \text{JERK_THRESHOLD}$ por 50ms consecutivos
PAUSA	Impulso finalizado	Calcula $ v_f $, θ , d , E_k , P . Guarda en RAM. LED azul 3 destellos. Espera 3s no-bloqueante.	Fin de espera de 3s → vuelve a IDLE

Diagrama de flujo del algoritmo:

```
[INICIO] → Setup: Wire.begin(SDA=6, SCL=7) · BNO055.begin() · SD.begin() · WiFi.softAP()  
↓  
[LOOP] → ¿Estado actual?  
■  
■ ■ IDLE: Lee IMU → Calcula J(t) → ¿ $|J| > \text{JERK\_THRESHOLD}$ ? → SÍ → Estado = IMPULSO  
■  
■ ■ IMPULSO: Lee IMU → Integra  $v_x += a_x \cdot \Delta t$  ;  $v_y += a_y \cdot \Delta t$  ;  $v_z += a_z \cdot \Delta t$   
■ → Guarda muestra en buffer RAM → ¿Jerk bajó? → SÍ → Estado = PAUSA  
■  
■ ■ PAUSA: Calcula  $|v_f| \cdot \theta \cdot d \cdot E_k \cdot P$  → Guarda en historial RAM  
■ → LED 3 destellos → millis() no-bloqueante 3000ms → Estado = IDLE  
↑ _____ ↓  
[Servidor web responde via callbacks – NO delay()]
```

2. ARQUITECTURA DE FIRMWARE — VARIANTES

El proyecto incluye dos variantes de firmware compilables de forma independiente:

VARIANTE	ARCHIVO	MODO DE OPERACIÓN	CASO DE USO
LANCEA_WiFi	LANCEA_WiFi.ino (v7.0)	Hotspot WiFi autónomo. Servidor web en 192.168.4.1. Gestión de atletas. Descarga CSV desde navegador.	Competencia y entrenamiento en campo sin PC.
LANCEA_NoSD	LANCEA_NoSD.ino	Conexión directa USB al PC. Telemetría serial ASCII en tiempo real. Sin WiFi. Sin SD.	Laboratorio, desarrollo, debugging con App Python.

Parámetros de configuración del firmware:

PARÁMETRO	VALOR DEFAULT	DESCRIPCIÓN / EFECTO
JERK_THRESHOLD	50.0 m/s³	Umbral de jerk para detectar inicio de lanzamiento. Reducir si no detecta; aumentar si hay falsas detecciones.
ARMING_ACCEL	15.0 m/s²	Aceleración mínima adicional para confirmar impulso real (evita detecciones por traspíe).
ALPHA_FILTER	0.30	Coefficiente del filtro exponencial de primer orden. Valores menores = más suavizado.
SAMPLE_INTERVAL_MS	10	Intervalo de muestreo en ms (100 Hz). Modificar requiere recalibración.

PARÁMETRO	VALOR DEFAULT	DESCRIPCIÓN / EFECTO
DEBUG_MODE	0	0 = producción. 1 = muestra Jerk en tiempo real en Monitor Serial para calibración.
MAX_LAUNCHES	100	Máximo de lanzamientos almacenados en RAM por sesión.
PAUSE_DURATION_MS	3000	Duración de la pausa entre lanzamientos (ms). No-bloqueante.

3. PROCEDIMIENTO DE CALIBRACIÓN DE JERK_THRESHOLD

La calibración garantiza que el sistema detecte lanzamientos reales y rechace movimientos no deseados (caminar, sacudidas accidentales).

CONDICIÓN	JERK TÍPICO (m/s³)	ACCIÓN REQUERIDA
Sensor en reposo sobre la mesa	< 5	—
Caminar normal con la jabalina	5 – 20	JERK_THRESHOLD debe estar por encima de este rango
Sacudida fuerte de escritorio	20 – 40	Zona de transición — calibrar según atleta
Lanzamiento real de jabalina	50 – 150	JERK_THRESHOLD en la mitad inferior (35–60)

Procedimiento: (1) Activar DEBUG_MODE 1 y recompilar. (2) Conectar Monitor Serial 115200 baud. (3) Ejecutar movimientos de referencia y anotar Jerk. (4) Establecer JERK_THRESHOLD = 60–70% del Jerk mínimo de lanzamiento real. (5) Activar DEBUG_MODE 0 y recompilar para producción.

4. GUÍA DE INSTALACIÓN DEL ENTORNO DE DESARROLLO

A. Configuración del ESP32-C3 en Arduino IDE

Prerrequisito: Arduino IDE 2.x con soporte ESP32 (Espressif Systems v2.0+). Agregar la siguiente URL en Archivo → Preferencias → URLs adicionales:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

LIBRERÍA	VERSIÓN MÍNIMA	FUENTE
Adafruit BNO055	≥ 1.6.3	Administrador de Librerías Arduino
Adafruit Unified Sensor	≥ 1.1.9	Administrador de Librerías Arduino
SD (built-in)	—	Incluida con ESP32 Espressif v2.0+
WiFi (built-in)	—	Incluida con ESP32 Espressif v2.0+
Wire (built-in)	—	Incluida con ESP32 Espressif v2.0+

B. Instalación de la App Python (lancea_app.py)

Requisito: Python 3.10+ con PATH habilitado.

```
pip install customtkinter pyserial pandas numpy matplotlib
```

Ejecutar con: **python lancea_app.py** | Windows: crear acceso directo .bat.

5. MÓDULOS DEL DASHBOARD PYTHON (lancea_app.py v3.0)

MÓDULO	FUNCIÓN PRINCIPAL	ENTRADAS / SALIDAS
Telemetría en Vivo	KPIs en tiempo real, coach virtual, consola serial, historial de sesión.	Entrada: Puerto COM serial · Salida: Reporte PDF en LANCEA_DATA/
Analizador de Logs	Carga archivos CSV/LOG. Gráficas matplotlib con estadísticas automáticas.	Entrada: .csv o .log · Salida: Gráficas interactivas
WiFi · Panel & Atletas	Tabla de lanzamientos, lista de atletas, badges de rendimiento.	Entrada: 192.168.4.1 · Salida: Importación CSV automática
SD · Volcado & Análisis	Descarga historial RAM del ESP32 via protocolo SD_DUMP.	Entrada: Comando DUMP serial · Salida: CSV + análisis
Gestión de Atletas	Registro, historial individual, carpeta por fecha.	Entrada: Formulario UI · Salida: LANCEA_DATA/atletas/

6. LISTA DE MATERIALES — BOM (Bill of Materials)

Materiales necesarios para ensamblar **una (1) unidad**. Precios en mercado colombiano local a febrero 2026.

ID	COMPONENTE	CA NT.	REFERENCIA / ESPECIFICACIÓN	COSTO COP
U1	Microcontrolador	1	XIAO ESP32-C3 (WiFi 2.4GHz, RISC-V 160MHz)	\$27.500
U2	IMU Sensor	1	Adafruit BNO055 o módulo genérico 9-DOF I²C	\$115.000
U3	Pantalla	1	OLED 0.91" I²C 128×32 (Blanco/Azul)	\$14.000
U4	Módulo MicroSD	1	Adaptador MicroSD interfaz SPI	\$8.500
U5	Memoria	1	MicroSD 16 GB Clase 10 FAT32	\$18.000
BAT 1	Batería	1	Li-Ion 14500 (3.7V, 800–1000 mAh)	\$20.000
IC1	Cargador	1	TP4056 con USB-C y protección de batería	\$3.500
SW1	Interruptor	1	Slide Switch SPDT Mini PCB Mount	\$800
BH1	Portapilas	1	Holder batería 14500 PCB Mount	\$2.500
B1	Pulsador	1	Tact Switch 6×6mm, 4 pines	\$500
D1	LED RGB	1	LED 5mm RGB cátodo común	\$1.000
R1–R3	Resistencias	3	220Ω, 10kΩ (1/4W)	\$200
PCB 1	Placa PCB	1	Baquelita Universal doble cara	\$6.000
M1	Filamento 3D	~20 g	PETG o PLA+ (chasis Sled)	\$5.000
M2	Varios insumos	1 lote	O-Rings, cable 30AWG, estaño	\$12.000

CATEGORÍA	ESTIMADO	REAL (Feb 2026)	VARIACIÓN
Electrónica y Sensores	\$188.000	\$184.500	-\$3.500
Potencia y Energía	\$25.000	\$26.800	+\$1.800
Mecánica e Insumos	\$21.700	\$24.000	+\$2.300

TOTAL POR UNIDAD	\$234.700	\$235.300	+\$600 (+0.26%)
------------------	-----------	-----------	-----------------

7. PLAN DE MANTENIMIENTO PREVENTIVO

ACTIVIDAD	FRECUENCIA	HERRAMIENTA	RESPONSABLE
Calibración JERK_THRESHOLD con atleta	Inicio de temporada o cambio de atleta	Monitor Serial Arduino IDE 2.x	Técnico firmware
Verificación O-Rings dentro de jabalina	Cada 50 lanzamientos o ruido	Inspección visual + agitación	Operador / Entrenador
Limpieza terminales batería y TP4056	Mensual	Alcohol isopropílico 99% + microfibra	Operador
Formateo y verificación MicroSD	Cada 6 meses o escritura lenta	PC + H2testw / badblocks	Técnico
Actualización de firmware	Según nueva versión en GitHub	Arduino IDE 2.x + USB-C	Técnico firmware
Inspección soldaduras PCB bajo lupa	Anual o tras impacto fuerte	Lupa 10x + multímetro	Ingeniero eléctrico
Recalibración completa BNO055 (NDOF)	Deriva de ángulo > ±3°	Procedimiento calibración Adafruit	Técnico firmware

8. DIAGNÓSTICO Y SOLUCIÓN DE PROBLEMAS TÉCNICOS

SÍNTOMA	DIAGNÓSTICO	SOLUCIÓN TÉCNICA
BNO055 no detectado — error I²C	Dirección I²C incorrecta o conexión SDA/SCL invertida	Verificar SDA=D4(GPIO6), SCL=D5(GPIO7). Probar dirección 0x29. Verificar 3.3V en VCC sensor.
Puerto COM no aparece en Arduino IDE	Driver USB-Serial no instalado en el PC	Instalar driver CH340 o CP2102. Verificar en Administrador de Dispositivos (Windows).
Datos se pierden al cerrar Arduino IDE	Señal DTR del IDE reinicia el ESP32 al cerrar la ventana	Solución hardware definitiva: condensador 10 µF entre pin RST y GND del XIAO ESP32-C3.
Jitter en muestreo ($\sigma > 1\text{ms}$ en CSV)	loop() bloqueado por escritura en SD Card	Implementar Timer de Hardware (ISR) para adquisición independiente. Buffer RAM para escritura en bloque.
Huecos en datos + ruido en línea VCC	Escritura muestra a muestra genera picos de corriente que afectan VCC del BNO055	Soldar capacitores 100µF (electrolítico) + 0.1µF (cerámico) en línea VCC del módulo SD.
Página web lenta / se cuelga durante grabación	delay() en el loop bloquea el servidor web	Reemplazar todos delay() por lógica millis() no-bloqueante. FSM garantiza esto en v7.0.

9. REFERENCIAS BIBLIOGRÁFICAS

[1] J. Campos y A. Brizuela, «Análisis biomecánico del lanzamiento de jabalina,» Biomecánica, vol. 18, no. 1, 2010.

[2] Espressif Systems, «ESP32-C3 Series Datasheet,» v1.4, 2023.

[3] Bosch Sensortec, «BNO055 Data Sheet,» Rev. 1.4, 2020.

[4] NXP Semiconductors, «I²C-Bus Specification,» UM10204, Rev. 7.0, 2021.

- [5] E. A. Lee y S. A. Seshia, Introduction to Embedded Systems, 2nd ed., MIT Press, 2017.
- [6] R. E. Mayagoitia et al., «Accelerometer and rate gyroscope measurement of kinematics,» J. Biomech., vol. 35, 2002.
- [7] J. Diebel, «Representing Attitude: Euler Angles, Unit Quaternions,» Stanford University, 2006.
- [8] S. Madgwick et al., «Estimation of IMU orientation using gradient descent,» IEEE ICORR, 2011.
- [9] World Athletics, «Technical Regulations 2024,» worldathletics.org

Documento elaborado por: Joseph Fernando Gutiérrez · Emerson Santiago Córdoba · Johan Rodrigo Piedrahita | Universidad del Quindío · Ingeniería Electrónica · Armenia, Colombia | 2025